

C++ OBJECT ORIENTED PROGRAMMING

Joel FALCOU



Object Oriented Paradigm

Toward Object Oriented Programming

The Aggregate Model

- An aggregate is the fundamental data structure in C++.
- It aggregates **members** of arbitrary types and name.
- Main way to add new types to the language.

```
1  struct point
2  {
3      double x;
4      double y;
5  };
6
7  struct line
8  {
9      point first, last;
10 };
```

Toward Object Oriented Programming

Limitations of the Aggregate Model

- An Aggregate is a simple batch of data.
- There is no intrinsic way to handle an aggregate's **coherence**.
- **Aggregates have no Invariants.**

From struct to class

- A **class** is a **service** provider.
- It happens to use data members to represent its internal state.
- **Classes 1st mission is to protect their coherence through Invariants.**

Toward Object Oriented Programming

From struct to class

- A **class** is a **service** provider.
- It happens to use data members to represent its internal state.
- **Classes 1st mission is to protect their coherence through Invariants.**

Object Oriented Programming in a nutshell

- Problems are solved by orchestrating calls to services.
- Services require complex states to be delivered.
- Classes **encapsulates** the required data to provide a service.
- Classes can be **refined** via **inheritance**.

Toward Object Oriented Programming

Encapsulation

- Classes can hide their data member from the user side of code.
- Members have a new properties: **access modifiers**.
- Classes relies on **constructors** to preserve invariants.

Polymorphism

- Classes behavior can be refined.
- Refined behaviors are selected at runtime depending on the actual type of a class.
- Relies on **inheritance**.
- Relies on **virtual functions**.

Encapsulation

Invariants

- Invariants are properties of a class that **must not change** during an instance lifetime.
- Calling a member function on an instance with invalid invariant is **Undefined Behavior**
- When an instance of a given class does not satisfy its invariant, it is **incoherent**.

Langage Support

- Access modifier: `public`, `private` and `protected`.
- Asserts
- Constructors

When a member should be **public**?

- When it is not involved in the class' invariant.
- The semantic of the class requires it.

When a member should be **private**?

- When it is involved in the class' invariant.
- It is only used in internal code from the class.

Rules of Thumb

- If no invariant exists, use **public** as much as possible.
- At the first glimpse of an invariant, use **private**.

Encapsulation

Access Modifiers - Example

```
1  struct point
2  {
3      // A point can be anywhere - no invariant
4      double x, y;
5  };
6
7  struct unit_point
8  {
9      // A unit_point MUST BE with [-1,1] x [-1,1]
10     double x, y;
11 };
```

Encapsulation

Access Modifiers - Example

```
1  struct unit_point
2  {
3      // We need *getters* to read x and y from client's code
4      double X() const { return x; }
5      double Y() const { return y; }
6
7      private:
8      // A unit_point MUST BE with [-1,1] x [-1,1]
9      double x, y;
10 };
```

Encapsulation

Access Modifiers - Example

```
1  struct unit_point
2  {
3      // How to prevent v to be outside the range of the invariant ?
4      void X(double v) { x = v; }
5      void Y(double v) { y = v; }
6
7      double X() const { return x; }
8      double Y() const { return y; }
9
10     private:
11         // A unit_point MUST BE with [-1,1] x [-1,1]
12         double x, y;
13 };
```

Encapsulation

Access Modifiers - Example

```
1  #include <cassert>
2
3  struct unit_point
4  {
5      // assert verify a predicates at runtime and terminate the program if it is false
6      void X(double v) { assert(v ≥ -1. && v ≤ 1.); x = v; }
7      void Y(double v) { assert(v ≥ -1. && v ≤ 1.); y = v; }
8
9      double X() const { return x; }
10     double Y() const { return y; }
11
12     private:
13         // A unit_point MUST BE with [-1,1] x [-1,1]
14         double x, y;
15 };
```

Encapsulation

Access Modifiers - Example

```
1  #include <cassert>
2
3  struct unit_point
4  {
5      void X(double v) { assert(v ≥ -1. && v ≤ 1.); x = v; }
6      void Y(double v) { assert(v ≥ -1. && v ≤ 1.); y = v; }
7      double X() const { return x; }
8      double Y() const { return y; }
9      private:
10     double x, y;
11 };
12
13 int main()
14 {
15     unit_point p;
16     p.X(0.5);      // OK
17     p.Y(-3.5);     // CRASH
18 }
```

Consequences of designing with Invariants

- Getters/Setters usage is clumsy.
- What if the default state of an object is already violating its own invariant?
- Should invariants be verified at construction? at destruction?

Solution: Constructors

- Use **constructor(s)** to ensure invariant compliance by design.
- Simplify usage for clients.
- Require less code to be written to handle lifetime corner cases.

What is a Constructor ?

- Constructors are **special member** handling the beginning of an object's lifetime.
- Constructors allow for more complex initialisation than simple aggregation.
- Constructors are used to enforce invariants at initialisation.

Encapsulation

What is a Constructor ?

```
1  struct unit_point
2  {
3      unit_point();           // Default constructor
4
5      unit_point(double vx);  // Custom constructor #1
6      unit_point(double vx, double vy); // Custom constructor #2
7
8      void X(double v) { assert(v ≥ -1. && v ≤ 1.); x = v; }
9      void Y(double v) { assert(v ≥ -1. && v ≤ 1.); y = v; }
10     double X() const { return x; }
11     double Y() const { return y; }
12
13     private:
14     double x, y;
15 };
```


Encapsulation

Default constructor

```
1  struct unit_point
2  {
3      // Default constructor - initializes an object into a 'neutral' state
4      unit_point() : x{0.}, y{0.} {}
5
6      double X() const { return x; }
7      double Y() const { return y; }
8      private:
9      double x, y;
10 };
11
12 int main()
13 {
14     unit_point p; // call default constructor
15     std::cout << p.X() << ", " << p.Y() << "\n";
16 }
```

Encapsulation

Custom constructor

```
1  #include <cassert>
2
3  struct unit_point
4  {
5      unit_point(double vx, double vy) // We must check that invariants are verified
6      {
7          assert(vx ≥ -1. && vx ≤ 1.); x = vx;
8          assert(vy ≥ -1. && vy ≤ 1.); y = vy;
9      }
10     private:
11     double x, y;
12 };
13
14 int main()
15 {
16     unit_point p(0.6,-0.9); // Correct values
17     unit_point q(6.9,-9.3); // Incorrect values
18 }
```

Encapsulation

Delegating constructors

```
1  #include <iostream>
2  #include <cassert>
3
4  struct unit_point
5  {
6      unit_point(double vx, double vy) { /* .. same code .. */ }
7      unit_point(double vx) : unit_point(vx,0.) {}
8      private:
9          double x, y;
10 };
11
12 int main()
13 {
14     unit_point p(0.6);      // Correct values
15     unit_point q(6.9,-9.3); // Incorrect values
16 }
```

Polymorphism

Definition

- **Polymorphism** is the way behavior of functions can be changed depending on their parameters types.
- There are multiple flavors of **Polymorphism** : static, dynamic, ad hoc, universal.

Object Oriented Polymorphism

- Refine a class behavior by **sub-classing** it.
- **Inheritance** is the way to explicit those sub-class relationships.
- Notion of **substitution**.

Polymorphism

Inheritance Relationship

- C++ supports both private and public inheritance.
- Public Inheritance: Polymorphism support.
- Private Inheritance: Code refactoring and sharing.

Virtual functions

- Member functions can be **virtual** or **abstract**.
- Contrary to other OO languages, being virtual is opt-in in C++.
- The **virtual** keyword marks a member functions as supporting polymorphic calls.

Polymorphism

Polymorphism in Action

```
1  class point
2  {
3      public:
4          point(int dx = 0, int dy = 0) :x(dx), y(dy) {}
5
6          void display() const
7          {
8              std::cout << "{" << x << ", " << y << "}";
9          }
10
11         private:
12             int x,y;
13     };
```

Polymorphism

Polymorphism in Action

```
1  // class A : public B indicates A inherits from B publicly
2  //                               v
3  class weighted_point : public point
4  {
5      public:
6      weighted_point ( int dx = 0, int dy = 0, float dm = 1.f) : point(dx,dy), m(dm)
7      // We can call base class constructor here -----^
8      {}
9
10     void display() const
11     {
12         point::display(); // Calling base class member
13         std::cout << "@" << m << " ";
14     }
15
16     float m;
17 };
```

Polymorphism

Polymorphism in Action

```
1  class point
2  {
3      /* ... */
4  };
5
6  class weighted_point : public point
7  {
8      /* ... */
9  };
10
11 int main()
12 {
13     point          x{7, 9};
14     weighted_point y{8,3,25.28f};
15     x.display();
16     std::cout << "\n";
17     y.display();
18 }
```


Polymorphism

Polymorphism in Action

```
1  #include <iostream>
2
3  class point { /* ... */ };
4  class weighted_point : public point { /* ... */ };
5
6  void display( point const& p )
7  {
8      p.display();
9      std::cout << "\n";
10 }
11
12 int main()
13 {
14     display( point{7, 9} );
15     display( weighted_point{8,3,25.28f} );
16 }
```

Polymorphism

Enabling polymorphic calls

- By default, member functions are not polymorphic.
- The **virtual** keyword must be specified to trigger polymorphic behavior
- A class with *at least* one polymorphic member function is said to be *polymorphic* itself.

Managing polymorphism interactions

- **override**
- **final**

Polymorphism

Polymorphism in Action

```
1  class point
2  {
3      public:
4          point(int dx = 0, int dy = 0) :x(dx), y(dy) {}
5
6          virtual void display() const // This function now supports polymorphic calls
7          {
8              std::cout << "{" << x << ", " << y << "}";
9          }
10
11         private:
12             int x,y;
13     };
```

Polymorphism

Polymorphism in Action

```
1  class weighted_point : public point
2  {
3      public:
4          weighted_point ( int dx = 0, int dy = 0, float dm = 1.f) : point(dx,dy), m(dm)
5          {}
6
7          void display() const override
8          {
9              point::display();
10             std::cout << "@" << m << " ";
11         }
12
13         float m;
14     };
```

Abstract Functions

- Some virtual functions make no sense in the base class.
- We can define a virtual function with no body to serve as a service declaration only.
- A class with *at least* one abstract member function is said to be *abstract* itself.

Consequences

- Abstract class can't be instantiated as value.
- Sub-class **must** provide an implementation.
- Those implementation-less classes are called **interfaces**.

Polymorphism

Polymorphism in Action

```
1  struct displayable
2  {
3      virtual ~displayable() {}
4      virtual void display() const = 0;
5  };
6
7  class point          : public displayable { /* ... */ };
8  class weighed_point : public point { /* ... */ };
9
10 void display( displayable const& p )
11 {
12     p.display();
13     std::cout << "\n";
14 }
```

Managing Polymorphic Objects

Context

- Polymorphic behavior is activated on pointer or reference to polymorphic types.
- References: easy to use but can't be used in containers.
- Pointers: composable with sequences but error-prone

Solution

- Use RAII-enabled semantic rich pointers
- Rich pointer + Polymorphic pointer = Polymorphic pseudo-value
- Composable with sequence and easy to use

Managing Polymorphic Objects

Semantic-rich Pointer: `std::shared_ptr`

```
1  #include <memory>
2  #include <iostream>
3
4  int main()
5  {
6      auto p1 = std::make_shared<std::string>(4, 'o'); // Resource Acquisition
7      std::cout << *p1 << "\n";
8
9      {
10         std::shared_ptr<std::string> p2 = p1; // Resource sharing
11         p2->append(7, '!');
12     } // p2 releases its resource. Nothing happens.
13
14     auto p3 = p1; // Resource sharing
15     p1.reset(); // p1 releases its resource explicitly. Nothing happens.
16     std::cout << *p3 << "\n";
17 }
18 // p3 is destroyed. Resource is finally released.
```


Managing Polymorphic Objects

Semantic-rich Pointer: `std::unique_ptr`

```
1  #include <memory>
2  #include <iostream>
3
4  int main()
5  {
6      auto p1 = std::make_unique<std::string>(4, '-'); // Resource Acquisition
7      std::cout << *p1 << "\n";
8
9      // std::unique_ptr is not copyable to prevent pointer sharing.
10     // std::unique_ptr<std::string> p2 = p1;
11
12     auto p3 = std::move(p1);    // Resource Transfer
13     p3->append(7, '!');
14     std::cout << *p3 << "\n";
15 }
16 // p3 is destroyed. Resource is finally released.
```

Managing Polymorphic Objects

Base Code

```
1 struct interface
2 {
3     virtual void perform() const = 0;
4     virtual ~interface() {}
5 };
```

Managing Polymorphic Objects

Base Code

```
1  struct interface;
2
3  class A : public interface
4  {
5      std::string value;
6      public:
7      A(std::string const& s) : value(s) {}
8      void perform() const { std::cout << "A\n"; }
9  };
10
11 class B : public interface
12 {
13     double first, last;
14     public:
15     B(double f, double l) : first(f), last(l) {}
16     void perform() const { std::cout << "B:" << first << " → " << last << "\n"; }
17 };
```

Managing Polymorphic Objects

Manual Management

```
1  struct interface;
2  class A : public interface { /** .. **/ };
3  class B : public interface { /** .. **/ };
4
5  int main()
6  {
7      std::vector<interface*> contents;           // sequence of pointers
8
9      for(int i=0;i<10;++i)
10     {
11         if(rand() % 2) contents.push_back(new A("text")); // raw new calls
12         else           contents.push_back(new B(i, (rand() % 100)/100.));
13     }
14
15     for(auto p : contents) p->perform();         // Pointer syntax
16     for(auto p : contents) delete p;           // Manual lifetime management
17 }
```

Managing Polymorphic Objects

Semantic-Rich Management

```
1  struct interface;
2  class A : public interface { /** .. **/ };
3  class B : public interface { /** .. **/ };
4
5  int main()
6  {
7      std::vector<std::unique_ptr<interface>> contents;
8
9      for(int i=0;i<10;++i) // No raw new calls
10     {
11         if(rand() % 2) contents.push_back(std::make_unique<A>("text"));
12         else          contents.push_back(std::make_unique<B>(i, (rand() % 100)/100.));
13     }
14
15     for(auto const& p : contents) // by-ref retrieval as unique_ptr is non-copyable
16         p->perform();           // Pointer syntax
17 } // Lifetime management is automated
```

Managing Polymorphic Objects

Semantic-Rich Management

```
1  struct interface;
2  class A : public interface { /** .. **/ };
3  class B : public interface { /** .. **/ };
4
5  int main()
6  {
7      std::vector<interface_t> contents; // Value-like definition
8
9      for(int i=0;i<10;++i)
10     {
11         // No abstraction leak
12         if(rand() % 2)  contents.push_back(A::make("text"));
13         else            contents.push_back(B::make(i, (rand() % 100)/100.));
14     }
15
16     for(auto const& p : contents)
17         p->perform();
18 }
```

Wrap Up - Object Oriented Paradigm

Wrap Up - Object Oriented Paradigm

Objects imply Invariants

- OOP is designing class defined by their invariants.
- If no invariants are required, aggregates are enough.
- Use asserts in selected member functions and constructors.

Polymorphism

- Class behaviors can be extended through inheritance.
- Polymorphism is **opt-in** via the `virtual` keyword.

Lifetime

- References are fine for functions arguments
- Raw pointers are error-prone, prefer using Semantic-Rich Pointers

Alternatives to Polymorphism

Alternatives to Polymorphism

Resolving a virtual function call

- Polymorphic classes contain a pointer to a **virtual functions table**.
- When a polymorphic call is issued, we use this table to retrieve the correct function pointer.
- An indirect function to call is then issued.
- Each sub-class modify entries in its own `vtable` depending on its set of overrides.

Consequences

- A polymorphic object is `sizeof(void*)` larger than expected.
- A polymorphic call implies pointer indirections and indirect call.
- All in all, a polymorphic calls can cost 10 to 30 extra cycles.

Alternatives to Polymorphism

Analyse the needs

- Do we know the concrete type of our object at all time?
- What is the size of the derived class set?
- Is the derived class set open ended?
- Is there any benefit to an intrusive interface?

Alternatives to consider

- Overloaded or generic functions
- `std::variant`
- CRTP
- Opaque Type

Alternatives to Polymorphism

Original Base Code

```
1  struct shape
2  {
3      virtual void display() const = 0;
4  };
5
6  class point      : public shape { void display() const override { /* ... */ } };
7  class line       : public shape { void display() const override { /* ... */ } };
8  class triangle   : public shape { void display() const override { /* ... */ } };
9  class circle     : public shape { void display() const override { /* ... */ } };
10
11  auto p = std::make_unique<triangle>( /* ... */ );
12  p->display();
```

Function based Alternative

This is a valid solution when:

- The derived class set is small and/or closed.
- The interface “surface” is limited.

```
1  struct point      { /* ... */ };
2  struct line       { /* ... */ };
3  struct triangle   { /* ... */ };
4  struct circle     { /* ... */ };
5
6  void display(point const& );
7  void display(line const& );
8  void display(triangle const& );
9  void display(circle const& );
10
11 triangle p{/*...*/};
12 display(p);
```

Function based Alternative

From overloads to generics

- If keeping the member function inside derived types is required, the polymorphic call can then be implemented as a function template.
- Allow for factoring the common behavior inside the template function.

```
1  struct point      { void display(); };
2  struct line       { void display(); };
3  struct triangle   { void display(); };
4  struct circle     { void display(); };
5
6  template<typename Shape>
7  void display(Shape const& s)
8  {
9      s.display();
10 }
11
12 triangle p{/*...*/};
13 display(p);
```

Function based Alternative

From generics to constrained generics (C++20)

- Use **Concepts** to define a compile-time set of syntax constraints the type passed to the generic function will have to fulfill.
- Prevent large, cryptic error if an unexpected type is erroneously passed to the function template
- Provide an “interface” like entity to document and refer users to.

```
1  template<typename T>
2  concept shape = requires(T const& t) { { t.display() } };
3
4  struct point      { void display(); };
5  struct line       { void display(); };
6  struct triangle   { void display(); };
7
8  template<shape S>
9  void display(S const& s)
10 {
11     s.display();
12 }
```

Variant

This is a valid solution when:

- The derived class set is closed.
- There is still a need for dynamic dispatching.

```
1  #include <variant>
2
3  struct shape
4  {
5      template<typename Shape> shape(Shape const& s) : storage(s) {}
6
7      void display() const { std::visit( [](auto const& s) { s.display() }, storage ); }
8
9      private:
10         std::variant<point,line,triangle,circle> storage;
11     }
12
13     shape p = triangle{/*...*/};
14     p.display();
```


This is a valid solution when:

- The interface “surface” is limited.
- The dispatch context is often static yet unspecified.

```
1  template<typename S> struct shape
2  {
3      void display() const { self().display(); }
4      protected:
5      S& self() const { return static_cast<S const&>(*this); }
6  };
7
8  struct point : public shape<point> // Recursively inherits from shape<point>
9  {
10     void display() const { /* ... */ }
11 };
12
13 template<typename S>
14 void do_something( shape<S> const& s ) { s.display(); }
```

Opaque Type

Main changes

- Polymorphic aspect is viewed as an implementation detail.
- Subscription to an interface needs not be non-intrusive.

```
1 struct shape
2 {
3     /* ??? */
4     void display() const { target->display(); }
5
6     displayable() = default;
7     template<typename T> displayable(T o) : target(new model<T>(o)) {}
8
9     private:
10     std::unique_ptr</* ??? */> target;
11 };
```

Opaque Type

Main changes

- Polymorphic aspect is viewed as an implementation detail.
- Subscription to an interface needs not be non-intrusive.

```
1  struct shape
2  {
3      struct interface
4      {
5          virtual void display() const = 0;
6          virtual ~interface() {}
7      };
8
9      template<typename T> struct model : interface
10     {
11         model(T t) : object(std::move(t)) {}
12         void display() const override { object->display(); }
13         T object;
14     };
15 }
```

Opaque Type

Main changes

- Polymorphic aspect is viewed as an implementation detail.
- Subscription to an interface needs ot be non-intrusive.

```
1  struct shape
2  {
3      /* ... */
4  };
5
6  // No inheritance - Direct usage is efficient
7  struct point    { void display(); { /* ... */ } };
8  struct line     { void display(); { /* ... */ } };
9  struct triangle { void display(); { /* ... */ } };
10
11 // Shape has a value-like semantic
12 std::vector<shape> ds = { point{}, line{}, triangle{}}
```

Wrap Up - Alternatives to Polymorphism

Wrap Up - Alternatives to Polymorphism

The Cost of Polymorphism

- Being polymorphic means being larger than usual.
- Virtual function calls are slower than usual.
- The cost/benefit of polymorphism may need to be assessed.

Static solutions

- Function templates and overload are to be considered.
- CRTP gives a polymorphic UX but with static dispatch.

Dynamic solutions

- `std::variant`: good for small/fixed size derived types set
- Opaque type: provide non-intrusive subscription to interface.

Design Principle

Common Sense Principle

Keep It Stupid Simple (KISS)

- Between two design strategies, use the less complex.
- Complexity is great at hiding bugs.

Don't Repeat Yourself (DRY)

- Code duplication is technical debt.
- Refactoring should often frequently but only if there is a need (WET).

You Aint Gonna Need it (YAGNI)

- Overgeneralisation a priori is a bad strategy.
- Let code grows organically from pragmatic needs.

The DEMETER Law

- All member functions M from an object O should limit themselves to call member functions from:
 - O itself
 - Parameters of M
 - Objects instantiated by M
 - Data member of O
- Chaining calls of functions member from object being the result of a call of functions member from object being the result of a ... is to be limited or seen as a review point.

Don't Do This At Home

```
return data.thing().member.do_it().x.print();
```

Coupling and Cohesion

Coupling

- Coupling measures the degree of interdependence between software components
- How closely are components related?
- How strongly are components related?

Cohesion

- Cohesion refers to the degree to which the elements inside a component belong together.
- It can be seen as a measure of the strength of relationship between the class's methods and data.

Coupling and cohesion (Larry Constantine, 1974)

- Low coupling often correlates with high cohesion.
- High coupling often correlates with low cohesion.

Procedural Coupling

- **Content coupling** occurs when one module uses the code of another module
- **Common coupling** occurs when several modules have access to the same global data.
- **External coupling** occurs when two modules share an externally imposed data format or protocol.

Object Oriented Coupling

- **Data coupling** occurs when modules share data through parameters.
- **Subclass coupling** is the de facto relationship between a class and its base class.
- **Temporal coupling** is the fact element of a module are together because they happen at the same time.

Coupling and Cohesion

Low Level of Cohesion

- **Coincidental cohesion:** parts of a component are grouped arbitrarily (e.g., “Utilities” class).
- **Logical cohesion:** parts of a component are grouped because they are logically doing the same thing (e.g, an “IO” class).

Medium Level of Cohesion

- **Temporal cohesion:** parts of a component are grouped by when they are processed.
- **Procedural cohesion:** parts of a component are grouped because they always follow a certain sequence of execution.
- **Communicational cohesion:** parts of a component are grouped because they operate on the same data.

Coupling and Cohesion

Medium Level of Cohesion

- **Temporal cohesion**: parts of a component are grouped by when they are processed.
- **Procedural cohesion**: parts of a component are grouped because they always follow a certain sequence of execution.
- **Communicational cohesion**: parts of a component are grouped because they operate on the same data.

High Level of Cohesion

- **Sequential cohesion**: parts of a component are grouped because the output from one part is the input to another.
- **Functional cohesion**: parts of a component are grouped because they all contribute to a single well-defined task of the component.
- **Atomic cohesion**: no part of the component can't be removed without making the component not working anymore.

The SOLID Design Philosophy

- **S**ingle Responsibility Principle
- **O**pen/Close Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation
- **D**ependency Inversion

Single Responsibility Principle

A class should have one and only one reason to change, so that a class should have only one job.

— Tom de Marco, 1979

What does this mean?

- Responsibility = specified behavior or service
- Classes that exposes unrelated services have multiple responsibilities.
- High coupling, Low cohesion.

Benefit

- Components complexity goes down.
- Limit physical coupling and improve cohesion.

Single Responsibility Principle

An Obvious SRP Violation

- This class allegedly takes care of uploading images.
- Then, why the dependency on Camera?
- Then, why providing `resizeImage`?

```
1 struct PhotoUploader
2 {
3     PhotoUploader(Camera& camera, CloudProvider& cloudProvider);
4     Image getImage();
5     Image resizeImage(int x, int y, const Image& image);
6     void uploadImage(Image& image);
7 };
```


Single Responsibility Principle

An Less Obvious SRP Violation

```
1  struct EmailSender
2  {
3      bool send(std::string const& customerID, std::string const& emailType )
4      {
5          Customer c = fetch(customerID);
6          auto contents = contents(c, emailType);
7          sender.send(contents);
8      }
9
10     std::string contents(Customer const& c, std::string const& emailType)
11     {
12         // ...
13     }
14
15     SMTPSender sender;
16 };
```

Single Responsibility Principle

Fixed Design

```
1  struct CustomerRepository
2  {
3      Customer fetch( std::string const& id );
4  };
5
6  struct EmailContentBuilder
7  {
8      std::string generate_contents( Customer const& c );
9  };
10
11 struct EmailSender
12 {
13     bool send( std::string const& to, std::string const& subject, std::string const& text );
14 };
```

Open/Close Principle

Software entities should be open for extension but closed for modification.

— Robert Martin, 1996

What does this mean?

- Software components can have their behavior changed or refined.
- Those changes should not require code source modification.

Benefit

- Less components to test.
- Less client code to recompile.
- Higher cohesion

Open/Close Principle

Sample OCP Violation

```
1  struct product_filter
2  {
3      std::vector<product> by_color( std::vector<product> const& in, Color c)
4      {
5          std::vector<product> out;
6          for(auto const& p : in) if(p.color == c) out.push_back(p);
7          return out;
8      }
9
10     std::vector<product> by_size( std::vector<product> const& in, std::size_t n)
11     {
12         std::vector<product> out;
13         for(auto const& p : in) if(p.size == n) out.push_back(p);
14         return out;
15     }
16 };
```

Fixed Design (1)

```
1  struct predicate { virtual bool check(product const&) const = 0; };
2
3  struct by_color : predicate
4  {
5      Color c;
6      bool check(product const&) const override { /* ... */ }
7  };
8
9  struct product_filter
10 {
11     std::vector<product> by( std::vector<product> const& in, predicate const& criterion)
12     {
13         std::vector<product> out;
14         for(auto const& p : in)
15             if(criterion.check(p)) out.push_back(p);
16         return out;
17     }
18 };
```

Fixed Design (2)

```
1  struct product_filter
2  {
3      template<typename Func>
4      std::vector<product> by( std::vector<product> const& in, Func criterion)
5      {
6          std::vector<product> out;
7          for(auto const& p : in)
8              if(criterion.check(p)) out.push_back(p);
9          return out;
10     }
11 };
12
13 product_filter f;
14 f.by(data, [](auto e) { return e.color == 'c'; });
```

Liskov Substitution Principle

Let $q(x)$ be a property provable about objects of x of type T . Then $q(y)$ should be provable for objects y of type S where S is a subtype of T .

— Barbara Liskov, 1999

What does this mean?

- If your code asks the concrete type of a polymorphic object, it is ill-formed.
- Derived classes can only loosen the pre-conditions of their member functions.
- Derived classes can only reinforce the post-conditions of their member functions.
- Derived classes must preserve their base class's invariants.

Liskov Substitution Principle

Example

```
1  struct rectangle
2  {
3      virtual void set_h(double hh) { h = hh; }
4      virtual void set_w(double ww) { w = ww; }
5      double area() const { return h*w; }
6
7      private:
8      double h,w;
9  };
10
11 struct square : public rectangle
12 {
13     void set_h(double hh) override { rectangle::set_h(hh); rectangle::set_w(hh); }
14     void set_w(double ww) override { rectangle::set_h(ww); rectangle::set_w(ww); }
15 };
```


Liskov Substitution Principle

Example

```
1  struct rectangle { /* ... */ };
2  struct square : public rectangle { /* ... */ };
3
4  void f(rectangle& r)
5  {
6      r.set_h(20);
7      r.set_w(10);
8      assert( r.area() == 200 );
9  }
10
11 int main()
12 {
13     rectangle r;
14     square    s;
15     f(r);
16     f(s);
17 }
```

Liskov Substitution Principle

Example

```
1  struct compute_area { virtual double area() const = 0; };
2
3  struct rectangle : public compute_area
4  {
5      rectangle(double hh, double ww) : h(hh), w(ww) {}
6      double area() const override { return h*w; }
7      private:
8      double h,w;
9  };
10
11 struct square : public compute_area
12 {
13     square(double ee) : edge(e) {}
14     double area() const override { return edge*edge; }
15     private:
16     double edge;
17 };
```

Interface Segregation

Many specific client interface are better than one general purpose interface.

— Robert Martin, 1996

What does this mean?

- Interfaces should not force client to implement unneeded member functions.
- Increase cohesion.
- Reduce coupling to at least Data coupling.

Sample IS Violation

```
1 struct behave_as_worker
2 {
3     virtual void work() = 0;
4     virtual void eat() = 0;
5 };
6
7 struct worker : behave_as_worker
8 {
9     void work() override { /* ... */}
10    void eat() override { /* ... */}
11 };
12
13 struct ten_x_worker : behave_as_worker
14 {
15     void work() override { /* ... */}
16     void eat() override { throw "Eating is for the weaks!"; }
17 };
18
19 void manage(behave_as_worker* p) { subordinate→work(p); }
```

Sample IS Violation

Fixed Design

```
1  struct behave_as_worker { virtual void work() = 0; };
2  struct behave_as_eater  { virtual void eat() = 0; };
3
4  struct worker : behave_as_worker, behave_as_eater
5  {
6      void work() override { /* ... */}
7      void eat()  override { /* ... */}
8  };
9
10 struct ten_x_worker : behave_as_worker { void work() override { /* ... */} };
11
12 void manage(behave_as_worker* p) { subordinate→work(p); }
```

Dependency Inversion

Entities must depend on abstractions, not on concretions.

High-level module must not depend on the low-level module, but on common abstractions.

— Robert Martin, 1996

What does this mean?

- Components must exchange data via abstractions.
- Avoid arbitrary “Tier based” design if they rely on concrete type.
- Better Coupling/Cohesion levels.

Dependency Inversion

Example

```
1  struct SqlConnection
2  {
3      bool connect() { /* ... */ }
4  }
5
6  class PasswordReminder
7  {
8      SqlConnection connection;
9
10     public:
11         PasswordReminder(SqlConnection const& c) : connection(c) {}
12 }
```

Dependency Inversion

Fixed Design

```
1  struct Connection { virtual bool connect() = 0;
2
3  struct SQLConnection : Connection
4  {
5      bool connect() override { /* ... */ }
6  }
7
8  class PasswordReminder
9  {
10     std::unique_ptr<Connection> connection;
11
12     public:
13     PasswordReminder(std::unique_ptr<Connection>&& c) : connection(std::move(c)) {}
14 }
```


Wrap Up - Design Principles

Wrap Up - Design Principles

Common Sense

- Those principles are the Occam's Razor of Software Design.
- Keep the code as simple and factored out as possible.
- When in doubt, choose the easy route.

SOLID

- Emanates from the Coupling/Cohesion spectrum.
- SOLID promotes actual agility.
- Code is more testable.
- Applicable to any kind of design even outside OOP.